

How sound moves the ink

Advection, pressure projection, and audio-driven dye injection.

06.01 · Dye is a tracer of motion

EXPLANATION

The visualizer pours colored dye into a moving velocity field. Audio energy drives injection; low and high frequency-band energy influence the forces. Touch supplies additional motion and dye.

The equations motivate an artistic incompressible-flow approximation. The simulation is not a water-surface model, and the visual patterns do not classify beats or genres.

Velocity and dye are separate fields. The visualizer uses a worker for motion and GPU-assisted dye rendering; the CPU fluid implementation makes the essential numerical operations explicit.

MATHS

Write velocity as $\mathbf{u}$, pressure as p , forcing as $\mathbf{f}$, and a dye channel as d . The model is

$$\begin{aligned}\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} &= -\nabla p + \mathbf{f}, & \nabla \cdot \mathbf{u} &= 0, \\ \partial_t d + \mathbf{u} \cdot \nabla d &= -\lambda d + s.\end{aligned}$$

Here s injects dye and λ controls fading. These are model equations; the app uses bounded grids and approximate numerical steps rather than an exact solution.

06.02 · Trace backward, then project

EXPLANATION

Semi-Lagrangian advection asks where a grid cell's contents came from. It traces backward through velocity and interpolates the previous field. This stays usable at interactive step sizes, although interpolation introduces numerical smoothing.

Pressure projection removes part of the velocity divergence. The CPU solver uses twelve pressure iterations and enforces zero normal velocity at the outer boundaries.

Finite iteration count, grid resolution, and boundary treatment all affect the result. A smooth-looking animation is not proof of an accurately resolved physical flow.

MATHS

For a field q , a backward trace gives

$$q^{\text{adv}}(\mathbf{x}) \approx q^n(\mathbf{x} - \Delta t \mathbf{u}^n(\mathbf{x})).$$

Bilinear interpolation samples off-grid locations. Projection is conceptually

$$\nabla^2 p = \nabla \cdot \mathbf{u}^*, \quad \mathbf{u}^{n+1} = \mathbf{u}^* - \nabla p.$$

The CPU code uses grid-scaled centered differences and a twelve-sweep pressure relaxation. It clamps Δt to $[0, 0.04]$ seconds and applies velocity drag $e^{-1.1\Delta t}$.

06.03 · Energy, persistence, and presentation

EXPLANATION

Sensitivity controls how strongly the audio injects motion and color; persistence controls how long the dye remains visible. Audio controls are eased rather than applied as abrupt jumps.

The CPU solver multiplies dye by a decay factor per simulation step. Its default decay is therefore sensitive to step frequency. Brightness controls presentation and does not change the audio measurement.

Distinguish a change in the sound from a change in the rendering settings before interpreting a more intense visual pattern.

MATHS

A simple energy-control mapping shown in the app's maths panel is

$$E = \text{clamp}(g \text{ RMS}, 0, 1).$$

For per-step decay ρ , dye evolves approximately as

$$d^{n+1} = \rho d^{\text{adv}}.$$

At fixed step duration, its equivalent continuous rate is $\lambda = -\ln(\rho)/\Delta t$. The CPU pixel mapping compresses nonnegative dye concentration with

$$I = 255(1 - e^{-d}).$$

This maps concentration to display intensity; it is not a loudness scale.